151. Reverse Words in a String

Medium | Topics | Companies

Given an input string `s`, reverse the order of the **words**.

A **word** is defined as a sequence of non-space characters. The **words** in `s` will be separated by at least one space.

Return *a string of the words in reverse order concatenated by a single space.*

Note that `s` may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:

```
Input: s = "the sky is blue"
Output: "blue is sky the"
```

Example 2:

```
Input: s = "  hello world  "
Output: "world hello"
Explanation: Your reversed string should not contain leading or trailing spaces.
```

Example 3:

```
Input: s = "a good   example"
Output: "example good a"
Explanation: You need to reduce multiple spaces between two words to a single space in the reversed string.
```

Constraints:

- `1 <= s.length <= 10^4`
- `s` contains English letters (upper-case and lower-case), digits, and spaces `' '`.
- There is **at least one** word in `s`.

RS / Day-87 / 3-11-26

Leetcode-151 : Reverse words in a string

You are given a string s containing multiple words separated by spaces.
Your task is to:-
→ Reverse the order of the words
→ Remove leading spaces
→ " trailing spaces
→ if multiple spaces exist between two words, replace them with just ONE space.

imp:- you are reversing the order of words, NOT reversing the chars inside each word.

example1 Input: s = "the sky is blue"
output: "blue is sky the".

example2 input: s = " Hello World "
O/P : "World Hello".

Approach => (Reverse the entire string).

Instead of storing words in a vector, we reverse the entire string first, then reverse each individual word.
We also remove unnecessary spaces while processing.

Step 1:- Reverse the entire string
suppose:- s = "the sky is pink"

after reversing the entire string:-

knip si yks eht.

the order of words is already reversed, but the characters inside each word are also reversed.

S2 → Reverse each individual word.
now reverse the characters of each word separately.

knip → pink
si → is
yks → sky
eht → the

Our result becomes:- ~~knis~~
pink is sky the
↳ required.

S3 → Remove xtra spaces,
while processing the string, we:
→ skip leading spaces
→ ignore repeated spaces
→ add only one space bet^n words.
→ avoid trailing spaces.

```
class solution{ public:
    string reverseWords(string s){
    reverse(s.begin(), s.end());
     int n = s.size();
```

int i=0; → pointer that reads characters.
int l=0; → " that writes characters into the processed string!

think of i as the reader & l as the writer.

```
while (i<n){
  while (i<n && s[i]==' '){
      i++;   if there are multiple spaces, keep moving i until you find a character.
  }
  if (i>=n) break;   if there are no more words, stop.
  if (l>0){          we want exactly ONE
    s[l++]=' ';      space between 2 words.
  }
```

l is our writing pointer. It tells us how many characters we have already written.
→ if l==0, we haven't written anything yet.
→ if l>0, we have already written at least one character (a word) so, if l>0, it means we are about to write another word and we need a space before it.

s[l++]=" means put a space at the current position of l. increase l by 1.

int start = i; remember where the word starts.
we need this position bcz we will need it for this particular word later.

we need to reverse the whole thing, so only is it soon.
when we encounter the 1st word, flip
initially, int i = 0;
we expect : int start = 1;

start = 4,

index: 0 1 2 3

K n i P

↑

i = 4.

while (i < n && s[i] != ' ') { → continue until the string is empty and the current char is NOT a space.
s[i++] = s[i++];
}

why? bcz we want to copy char until the current word ends.

K → copy
n → copy
i → copy
P → copy
space → stop.

we have now copied one complete word.

→ read the char at position i, copy it to position i, move both pointers one step forward.

reverse(s.begin() + start, s.begin() + i);
} now we reverse each individual word to restore its original spelling.

s.resize(0); → this removes all the unwanted characters remaining after the processed portion.

return s;

↳ return the answer.

t.c = O(n), s.c = O(1)

dry run: input: " a good example "

s1 → remove the entire spaces.

" elpmaxe doog a "

s2 → skip leading spaces,

our reading pointer i skips the spaces at the beginning.

it reaches the 1st char e.

s3 process each word

word encountered	after reversing the word	Answer
elpmaxe	example	example
doog	good	example good
a	a	example good a.

extra spaces are skipped, & only one space is inserted between words.

final output: "example good a".

Accepted 62 / 62 testcases passed

Akankshyaa submitted at Nov 03, 2026 09:36

Analysis Solution

Save 41%

Big Diwali Dhamaka

Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!

Runtime

1 ms | Beats 51.00%

Memory

9.82 MB | Beats 93.61%



Code | C++

```cpp
class Solution {
public:
    string reverseWords(string s) {
        reverse(s.begin(), s.end());
        int n= s.size();
        int i=0;
        int l=0;
        while(i<n){
```

View more

More challenges

- 186. Reverse Words in a String II

Write your notes here

```cpp
class Solution {
public:
    string reverseWords(string s) {
        reverse(s.begin(), s.end());
        int n= s.size();
        int i=0;
        int l=0;
        while(i<n){
            while(i<n && s[i]==' '){
                i++;
            }
            if(i>=n) break;
            if(l>0){
                s[l++]= ' ';
            }
        int start =l;
        while (i<n && s[i] != ' ') {
            s[l++]=s[i++];
        }
        reverse(s.begin()+start, s.begin()+l);
         }
         s.resize(l);
        return s;
    }
};
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s =

"the sky is blue"

Output

"blue is sky the"

Expected

"blue is sky the"